

# Algorithms

Lecture #05

**Syed Hamed Raza**

# 2-dimension Maxima Revisited

## ***2-dimension Maxima Revisited***

Recall the 2-d maxima problem:

- Let a point  $p$  in 2-dimensional space be given by its integer coordinates,  $p = (p.x, p.y)$ .
- A point  $p$  is said to dominated by point  $q$  if  $p.x \leq q.x$  and  $p.y \leq q.y$ .

# 2-dimension Maxima Revisited

## ***2-dimension Maxima Revisited***

- Given a set of  $n$  points,  $P = \{p_1, p_2, \dots, p_n\}$  in 2-space a point is said to be *maximal* if it is not dominated by any other point in  $P$ .
- The problem is to output all the maximal points of  $P$ .

# 2-dimension Maxima Revisited

## ***2-dimension Maxima Revisited***

- We introduced a brute-force algorithm that ran in  $\Theta(n^2)$  time.
- It operated by comparing *all pairs* of points.
- Is there an approach that is significantly better?

# 2-dimension Maxima Revisited

## ***2-dimension Maxima Revisited***

- The problem with the brute-force algorithm is that it uses no intelligence *in pruning* out decisions.
- For example, once we know that a point  $p_i$  is dominated by another point  $p_j$ , we do not need to use  $p_i$  for eliminating other points.

# 2-dimension Maxima Revisited

## ***2-dimension Maxima Revisited***

- This follows from the fact that dominance relation is *transitive*.
- If  $p_j$  dominates  $p_i$  and  $p_i$  dominates  $p_h$  then  $p_j$  also dominates  $p_h$ ;  $p_i$  is not needed.

# Plane-sweep Algorithm

## *Plane-sweep Algorithm*

- We will sweep a vertical line across the 2-d plane from left to right.
- As we sweep, we will build a structure holding the maximal points lying to the left of the sweep line.



# Plane-sweep Algorithm

## ***Plane-sweep Algorithm***

### Plane-sweep Algorithm

- When the sweep line reaches the rightmost point of  $P$ , we will have the complete set of maximal points.
- This approach of solving geometric problems is called the *plane sweep*.



# Plane-sweep Algorithm

## ***Plane-sweep Algorithm***

### Plane-sweep Algorithm

- Store the existing maximal points in a list.
- The points that  $p_i$  dominates will appear at the end of the list because points are sorted by x-coordinate.
- Scan the list left to right.

# Plane-sweep Algorithm

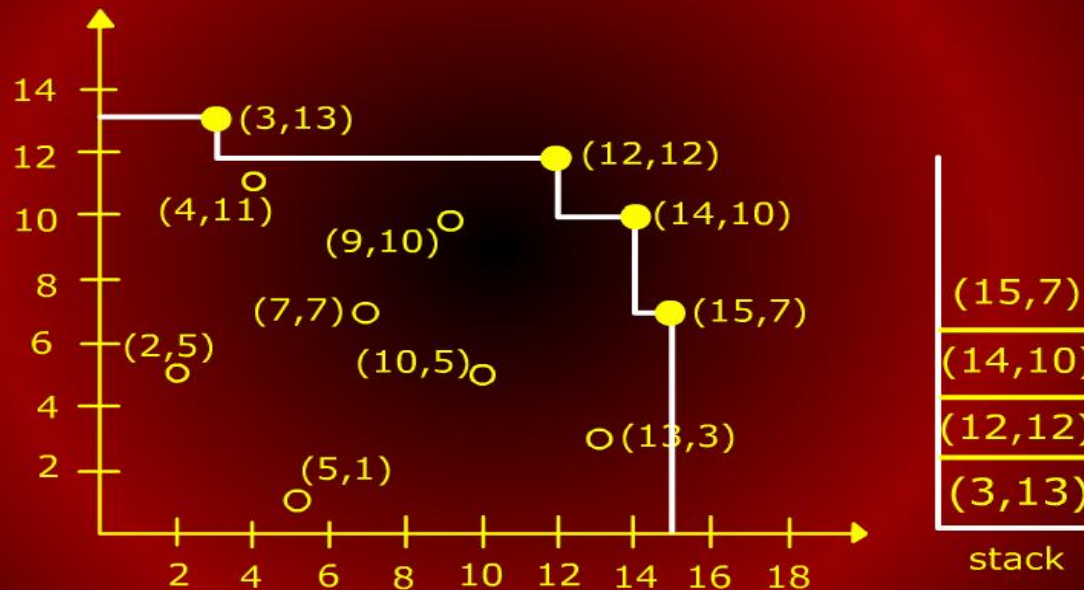
## ***Plane-sweep Algorithm***

Plane-sweep Algorithm:

- We add maximal points onto the end of a list and delete from the end of the list.
- We can thus use a *stack* to store the maximal points.
- The point at the top of the stack will have the highest x-coordinate.
- Here is the algorithm:

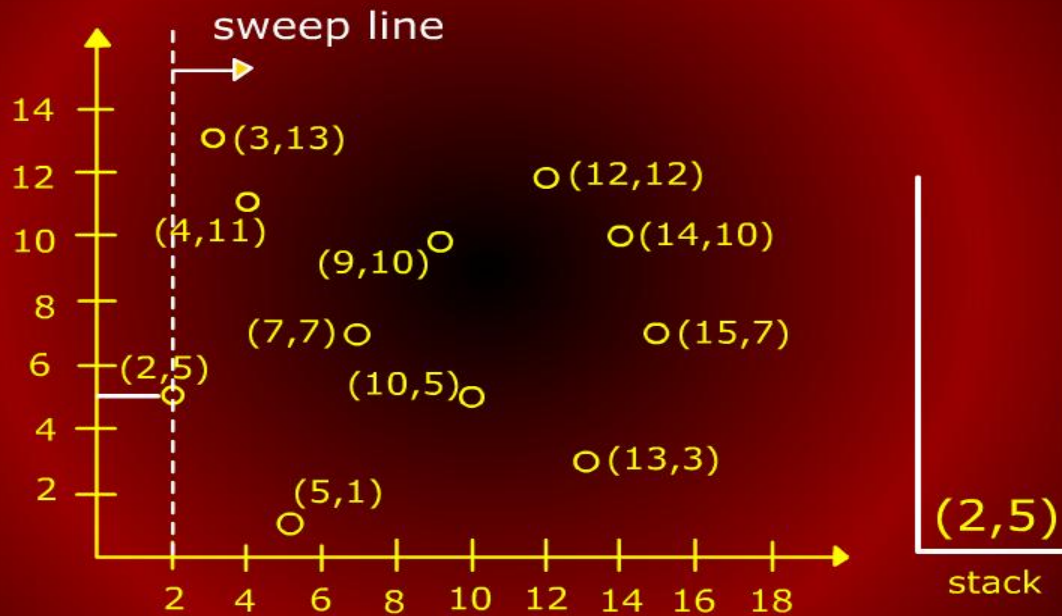
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



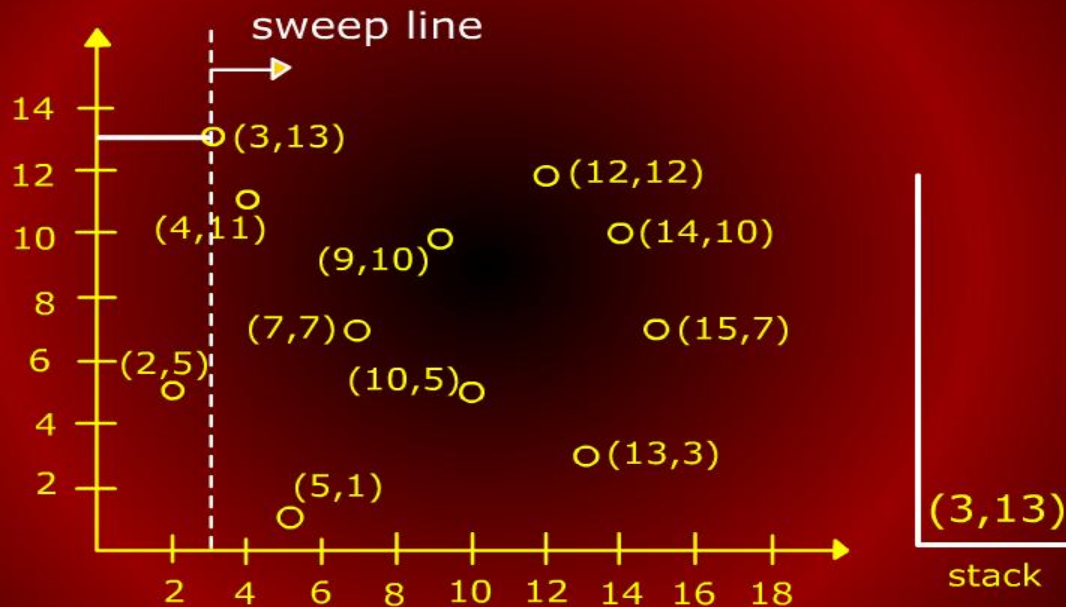
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



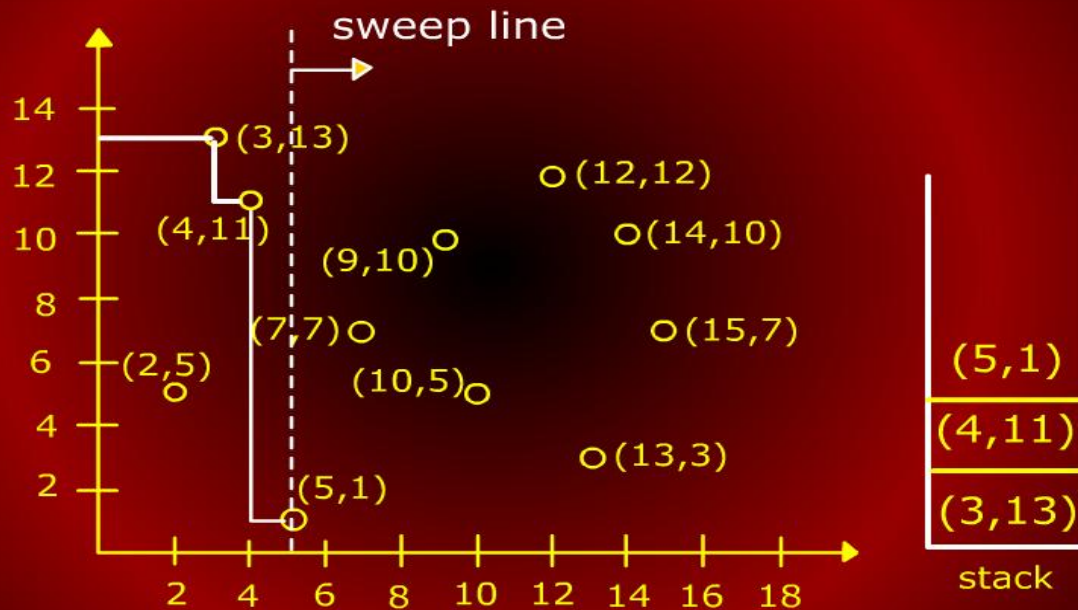
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



# Plane-sweep Algorithm

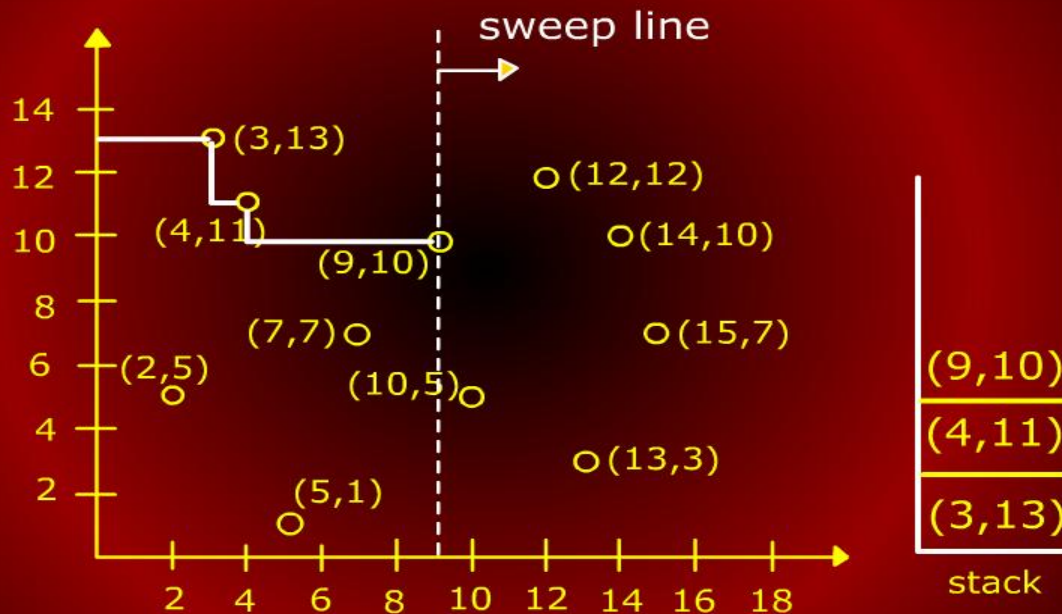
## *Plane-sweep Algorithm*





# Plane-sweep Algorithm

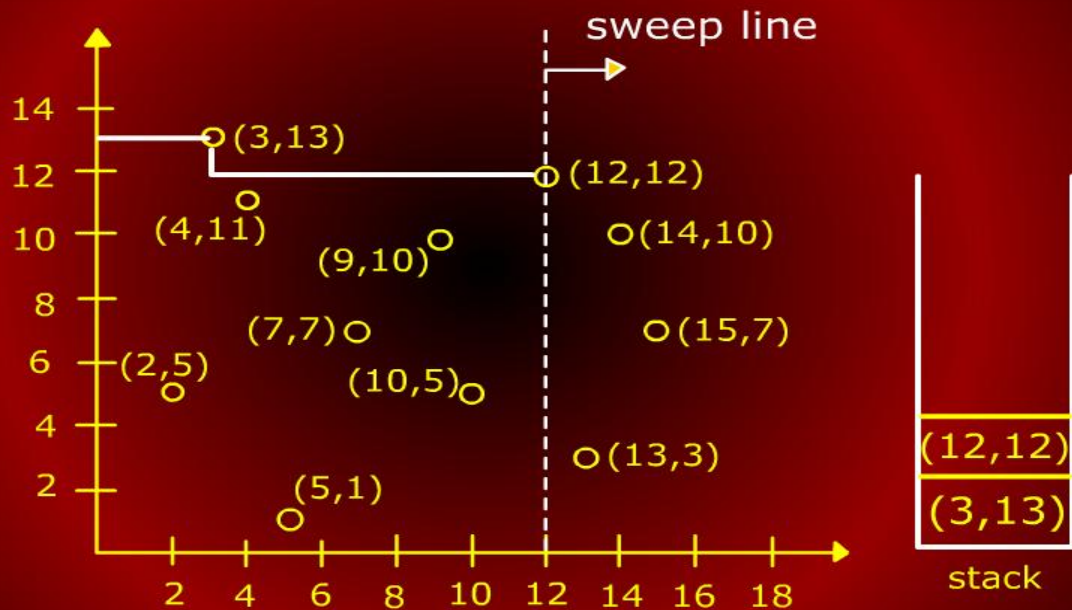
## *Plane-sweep Algorithm*





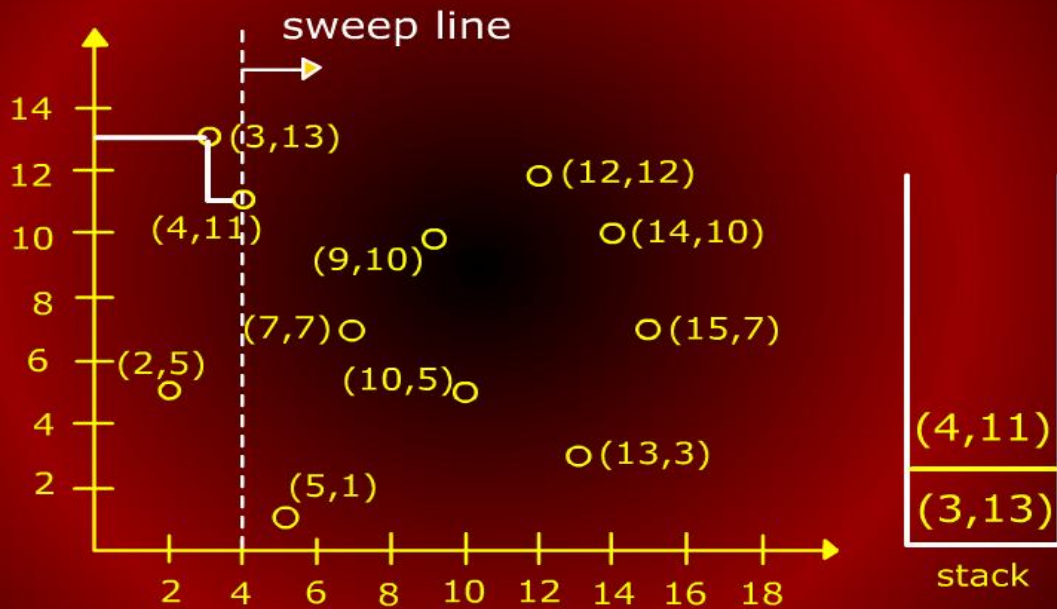
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



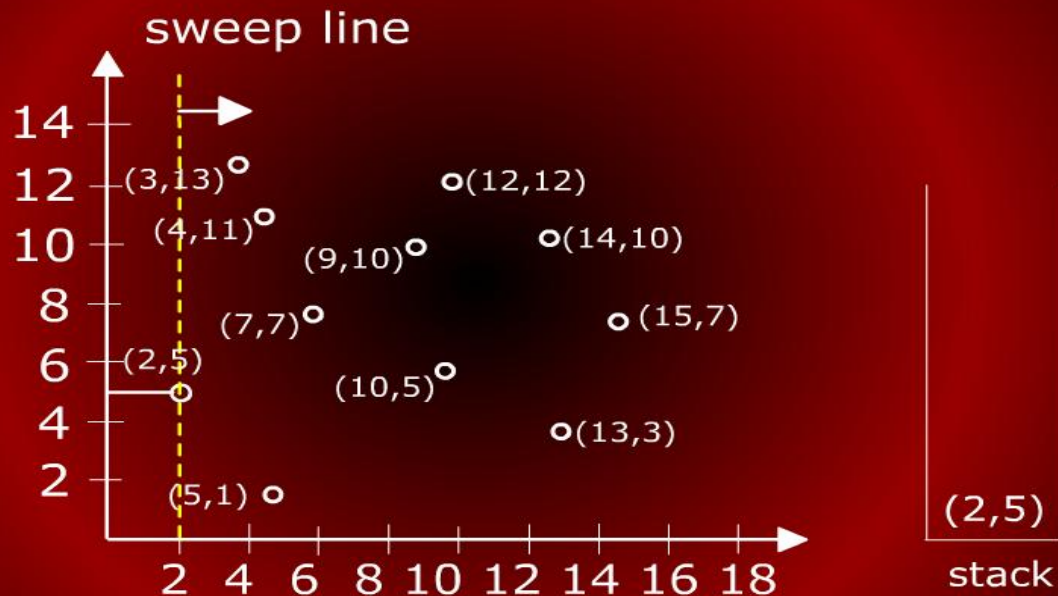
# Plane-sweep Algorithm

## ***Plane-sweep Algorithm***

```
PLANE-SWEEP-MAXIMA(n, P[1..n])  
1 sort P in increasing order by x;  
2 stack s;  
3 for i ← 1 to n  
4 do  
5   while(s.notEmpty() & s.top().y ≤ P[i].y)  
6     do s.pop();  
7   s.push(P[i]);  
8 output the contents of stack s;
```

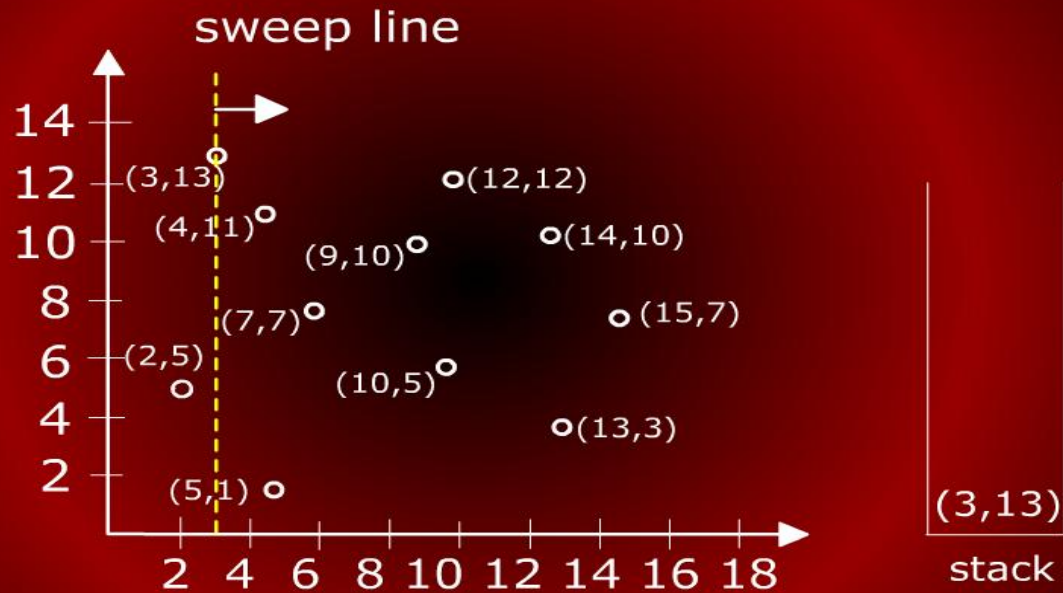
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



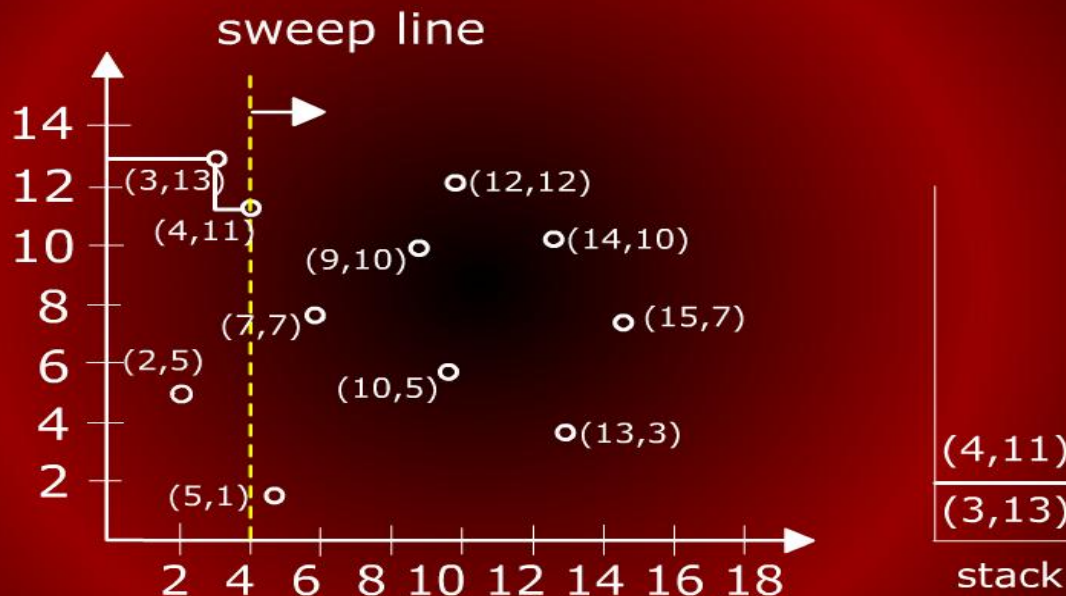
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



# Plane-sweep Algorithm

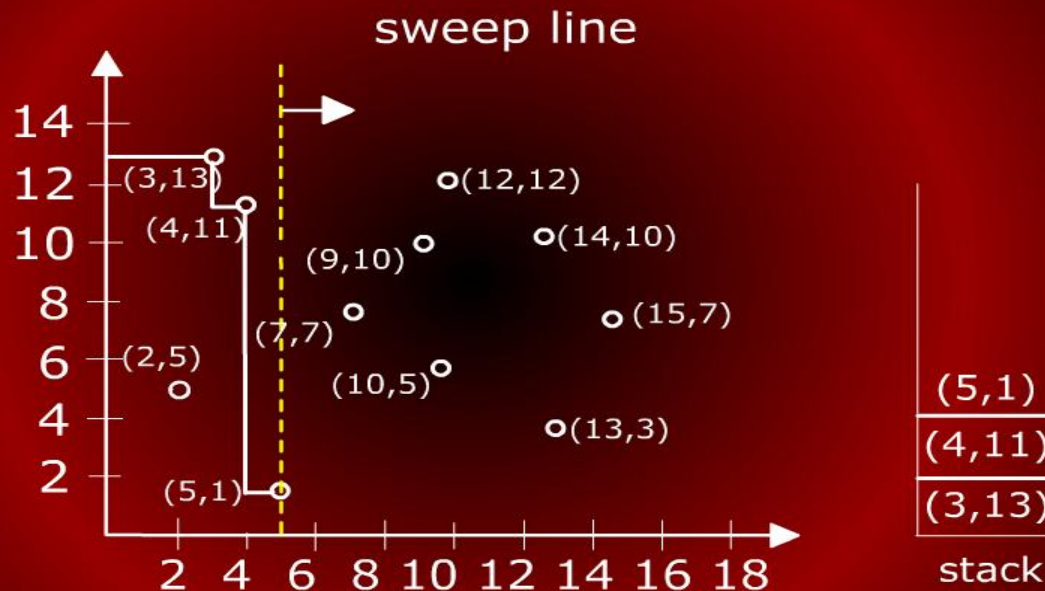
## *Plane-sweep Algorithm*





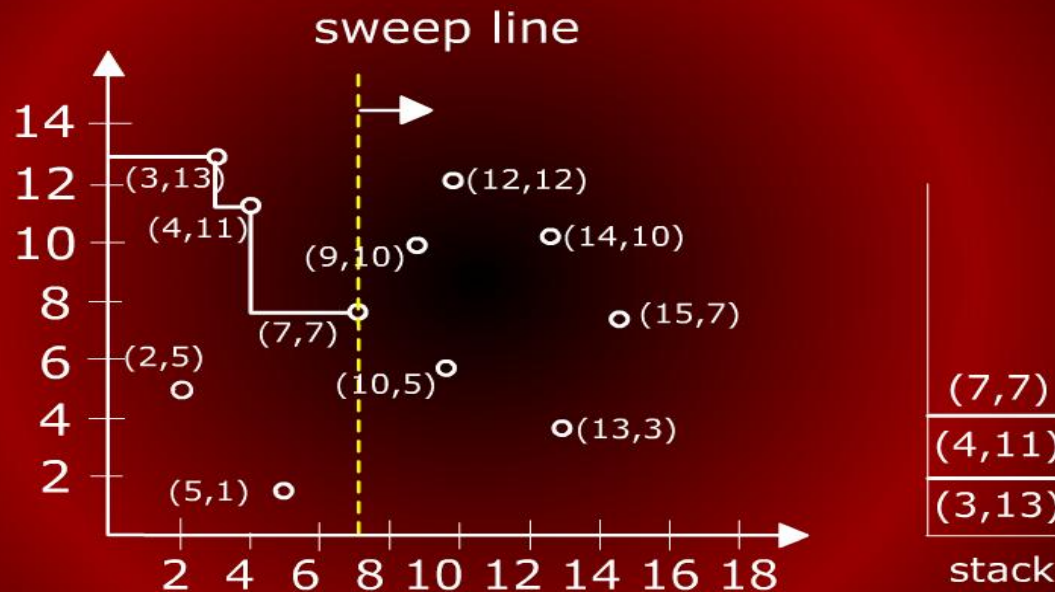
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



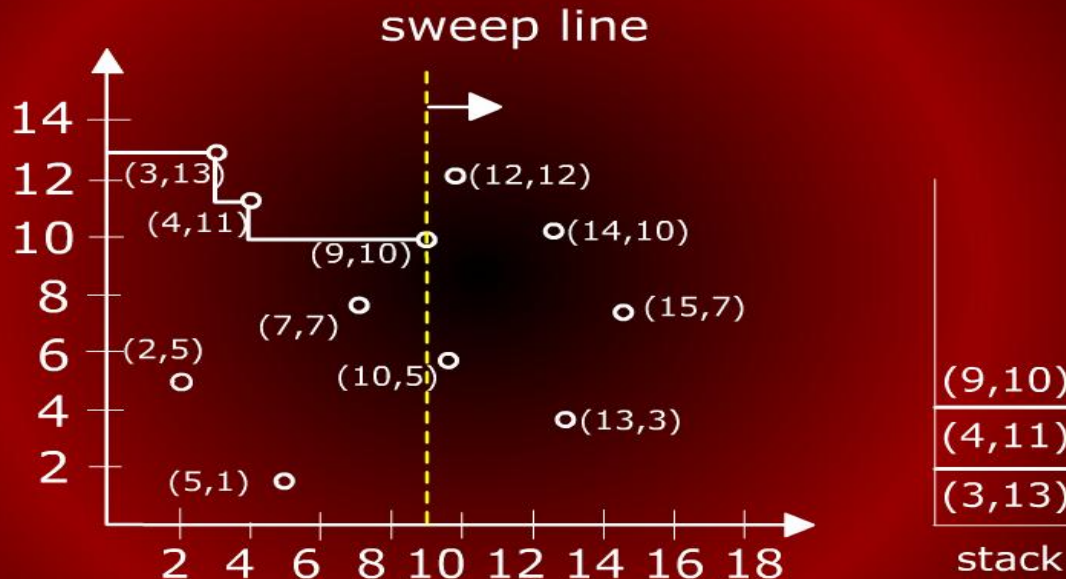
# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



# Plane-sweep Algorithm

## *Plane-sweep Algorithm*



# Analysis of Plane-sweep Algorithm

## ***Analysis of Plane-sweep Algorithm***

- Sorting takes  $\Theta(n \log n)$ ; we will show this later when we discuss sorting.
- The for loop executes  $n$  times.
- The inner loop (seemingly) could be iterated  $(n - 1)$  times.
- It seems we still have an  $n(n - 1)$  or  $\Theta(n^2)$  algorithm.

# Analysis of Plane-sweep Algorithm

## ***Analysis of Plane-sweep Algorithm***

- We pop an element off the stack each time we go through the inner while-loop.
- It is impossible to pop more elements than are ever pushed on the stack.



# Analysis of Plane-sweep Algorithm

## ***Analysis of Plane-sweep Algorithm***

- The for-loop iterates  $n$  times and the inner while-loop also iterates  $n$  time for a total of  $\Theta(n)$ .
- Combined with the sorting, the runtime of entire plane-sweep algorithm is  $\Theta(n \log n)$ .
- Therefore, the inner while-loop cannot execute more than  $n$  times over the entire course of the algorithm. (Make sure that you understand this).



# Comparison

## *Comparison*

- How much of an improvement is plane-sweep over brute-force?
- Ratio of running times:

$$\frac{n^2}{n \log n} = \frac{n}{\log n}$$

- Therefore, the inner while-loop cannot execute more than  $n$  times over the entire course of the algorithm. (Make sure that you understand this).

# Comparison

## *Comparison*

| n       | logn | $\frac{n}{\log n}$ |
|---------|------|--------------------|
| 100     | 7    | 15                 |
| 1000    | 10   | 100                |
| 10000   | 13   | 752                |
| 100000  | 17   | 6021               |
| 1000000 | 20   | 50171              |

For  $n = 1,000,000$ , if plane-sweep takes 1 second, the brute-force will take about 14 hours!.

# Analysis of Plane-sweep Algorithm

## ***Analysis of Plane-sweep Algorithm***

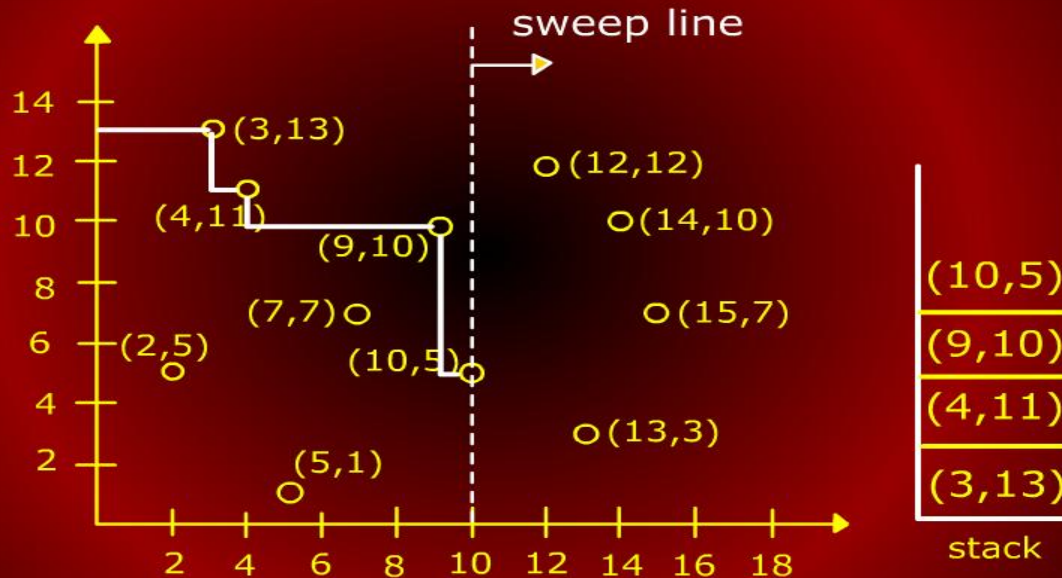
- We pop an element off the stack each time we go through the inner while-loop.
- It is impossible to pop more elements than are ever pushed on the stack.

# Design & Analysis of Algorithms

# 2-Dimension Maxima Revisited

# Plane-sweep Algorithm

## *Plane-sweep Algorithm*





# Analysis of Plane-sweep Algorithm

## ***Analysis of Plane-sweep Algorithm***

- The for loop executes  $n$  times.
- The inner loop (seemingly) could be iterated  $(n - 1)$  times.
- It seems we still have an  $n(n - 1)$  or  $\Theta(n^2)$  algorithm.
- Got fooled by simple minded loop-counting.

# Asymptotic Notation

## ***Asymptotic Notation***

- You may be asking that we continue to use the notation  $\Theta()$  but have never defined it.
- Let's remedy this now.
- Given any function  $g(n)$ , we define  $\Theta(g(n))$  to be a set of functions that *asymptotically equivalent* to  $g(n)$ .

# Asymptotic Notation- Example

## ***Asymptotic Notation - Example***

- For this, need to show that there exist positive constants  $c_1$  and  $n_0$ , such that  $f(n) \geq c_1 n^2$  for all  $n \geq n_0$ .
- Consider the reasoning
$$\begin{aligned}f(n) &= 8n^2 + 2n - 3 \geq 8n^2 - 3 \\ &= 7n^2 + (n^2 - 3) \geq 7n^2\end{aligned}$$
- Thus  $c_1 = 7$ .

